

Practical and Engineering Limitations

Despite its strengths, full integration of Isabelle/HOL was not feasible within the constraints of this undergraduate project.

The most significant limitation was architectural complexity. Isabelle is not designed as a lightweight embeddable library for client-side use. Instead, it operates as a sophisticated theorem proving environment requiring:

- A dedicated runtime
- Complex dependency management
- Server-side execution

Embedding Isabelle into a browser-based educational tool would require designing a backend bridge capable of:

- Translating user proof steps into Isabelle theory syntax
- Constructing valid proof scripts
- Executing them within a running Isabelle process
- Parsing and interpreting error outputs
- Returning structured feedback to the frontend

This integration layer would introduce substantial additional system complexity.

A second major constraint is deployment overhead. Running Isabelle in a production environment requires:

- Persistent server infrastructure
- Process isolation
- Resource management
- Protection against arbitrary code execution

Because users would effectively submit dynamically constructed proof content, careful sandboxing mechanisms would be required. Implementing secure sandboxing would significantly increase both development time and maintenance requirements.

Third, Isabelle proofs are typically structured as theory files or structured proof scripts rather than direct line-by-line natural deduction entries. Mapping an interactive, incremental proof interface onto Isabelle's internal representation would require a non-trivial translation framework. This would involve:

- Formal encoding of proof states
- Scope and assumption tracking compatibility
- Error message interpretation
- Synchronisation between frontend and backend states

Such a translation mechanism could constitute a major standalone project.

Additionally, responsiveness was a central non-functional requirement of the system. The implemented design emphasises immediate validation feedback. Introducing server-side theorem proving would introduce latency, dependency on network stability, and potential performance bottlenecks.

Under these constraints, implementing a local rule-validation engine provided sufficient correctness enforcement for propositional logic while maintaining architectural simplicity and usability.

Thus, while Isabelle/HOL offers superior formal guarantees, its integration would have shifted the project focus toward distributed theorem prover engineering rather than educational interface design, exceeding the intended scope.